

ORIGINAL ARTICLE

A fully convolutional neural network for fast detection, classification, and segmentation of fabric defects

Swash Sami Mohammed¹  · Hülya Gökulp Clarke¹ · Sarmad Nozad Mahmood²

Received: 9 October 2024 / Accepted: 3 July 2025 / Published online: 22 August 2025

© The Author(s), under exclusive licence to Springer-Verlag London Ltd., part of Springer Nature 2025

Abstract

The integration of artificial intelligence and image processing for fabric defect detection is gaining prominence due to its practical significance in enhancing production quality. This study proposes a fast and accurate convolutional neural network (CNN) designed to detect defects in fabric with minimal computational complexity. The model processes input images of size 256×256 and generates defect masks of size 64×64 . To improve detection accuracy, the model incorporates techniques such as ResNet, scheduled learning rate policies, data augmentation, and a weighted cross-entropy loss function. Trained on a diverse dataset of 2,681 defect samples from four fabric types and defect classes (holes, oil stains, color stains, and roller marks), the model achieved an accuracy of over 96%, a loss value below 0.1, and high recall, precision, and F1-Score. Compared to other state-of-the-art models, the proposed model delivers competitive performance with significantly faster prediction times, making it suitable for real-world fabric inspection applications.

Keywords Fabric defect detection · Data annotation · Defect segmentation and classification · Convolutional neural network (CNN) · Machine learning model · Artificial intelligence · Deep learning

1 Introduction

Identifying fabric imperfections is very important in the textile industry to ensure the quality and reliability of the textile product [1]. The fabric defect is irregularities or imperfections on the fabric material manifested as holes, stains, tears, snags, areas of different colors, or distortion in weaves that constitute the fabric's quality, durability, and performance. The existence of fabric defects usually leads to dissatisfaction on the part of the end customers and subsequent monetary losses to the manufacturers. Defect detection in fabrics has usually been carried out by human inspectors [2]. However, manual inspection is time-consuming and extremely subjective and is further susceptible to human errors and inconsistency.

With the rising demand for automated and objective methods of fabric defect detection, the last couple of decades has witnessed a great deal of research going into the application of machine learning and computer vision techniques for automated fabric defect detection systems [3]. Machine learning techniques, such as supervised learning, unsupervised learning, and deep learning [4, 5], have been deployed for the development of algorithms which can allow for the automatic detection of defective spots and the very accurate classification of fabric defects [6, 7]. Machine learning techniques for fabric defect detection typically consist of several steps, starting from data acquisition, which may include the capture of high-resolution images of fabric samples or rolls with the help of digital cameras or scanning devices [8]. The acquired images are used in a defect detection system. In addition, it is often necessary to perform some preprocessing step on the acquired images, such as resizing, color



normalization, noise removal, and enhancement, to ensure the quality and consistency of the input data for the subsequent processing [9]. The feature extraction process extracts the image features most salient to the differentiation for the training of some model which will be able to represent normal fabric regions differently from defective ones [10]. Examples of these features could be color, texture, shape, gradient, and statistical measures [11]. The feature vectors are then used in training process of machine learning model. The model will learn to discriminate normal regions from defective ones based on the training's evidence coming from labeled samples. Various pattern recognition techniques have been explored to enhance defect classification and detection accuracy in textile inspection [12–15]. When the model is trained, it is generally set in motion for the detection and classification of defects on new unseen fabric images. The trained model will analyze the features extracted from test images and predict whether each considered segmented region is normal or defective [16]. Certain methods also segment out defective areas, thus accurately localizing and delineating the defects. The performance of a fabric defect detection system is measured based on the parameters such as accuracy, precision, recall, and F1-score. Validation itself is based on either using ground truth annotations or on human expert judgment. It is noteworthy that there exist motivating and alternative methodologies concerning textile defect detection include localizers and classifiers. Such principles will also consider the uncertainties and imprecisions that naturally occur in textile inspection as they work toward adaptively localizing and classifying defects through fuzzy divergence computations [17].

Where bidirectional cuts have already reached the association stage, SSG2 has followed suit by developing its relatively sophisticated segmentation framework. SSG2 exploits the principles of self-supervised learning, harnessed with GANs, in order to output improved segmentation results in cases where only few noticed instances or poorly observed patterns are available. One other method of taking care of the susceptibility for the domain of fabric-deformation detection is systematically devising strong and trivial algorithms that digest late conventionally on datasets [18].

Machine learning algorithms for automatic detection of fabric defects contribute to enhanced inspection speed, improved accuracy, and reduced reliance on subjective judgments from human observers [19], thus allowing early detection and remediation of defects, improving the quality of the product, lowering production costs, and increasing customer satisfaction [20]. Researchers are still inventing new algorithms, models, and datasets to develop the field of fabric defect detection further. The building of robust, smart automated systems represents a considerable opportunity for optimizing manufacturing processes, minimizing waste, and delivering quality textile products throughout various sectors.

Recently, researchers have faced several difficulties when dealing with fabric defect detection. The most challenging problem has been the lack of a comprehensive, well-annotated, and standardized dataset. Due to limited and imbalanced data, most fabric defect datasets often suffer from a scarcity of labeled data [21, 22]. Acquiring and annotating a large number of fabric images with diverse defect types can be time-consuming and expensive. Additionally, fabric defects are typically rare compared to normal fabric regions, resulting in imbalanced class distributions. This class imbalance can affect the performance of machine learning models and bias them toward the majority class. On the other hand, annotating fabric defects is a subjective task, as different annotators may have varying interpretations of what constitutes a defect. This subjectivity can lead to inconsistencies and discrepancies in the ground truth annotations, affecting the reliability of the dataset. Since fabric defects exhibit significant variability in terms of types, sizes, shapes, colors, and textures, the collected dataset should encompass a wide range of defect variations to ensure that the trained models are robust and can handle different defect scenarios [23]. Another solution to this problem is to generate synthetic datasets; however, generating accurate defect textures, shapes, and variations that mimic real-world defects is a complex process [24]. Synthetic datasets may not fully capture the complexities and intricacies of actual fabric defects, leading to potential limitations in the model's generalization capability.

Addressing these problems, this work aims to collect a wide, well-organized, and professionally annotated fabric defect dataset that has been captured while fulfilling the required standards for this purpose. It will be made partially available for public use to benefit the research community and promote further advancements in the field.

Comprehensive documentation will also be provided, including information about the fabric types, defect types, annotation process, and any preprocessing steps applied to the collected dataset. In addition, a very fast and highly accurate CNN model, this model is proposed for detecting, classifying, and segmenting fabric defects. The proposed model is designed, optimized, and trained with special techniques to be fast and accurate simultaneously. The trained model has scored high on the metrics used, and promising results were obtained while testing the trained model. The key highlights and novel contributions of this paper include:

1. *Model Design*: A specially designed convolutional neural network (CNN) that reduces computational complexity while maintaining high accuracy.
2. *Techniques Used*: Innovative use of ResNet architecture, scheduled learning rate policies, data augmentation, and weighted cross-entropy loss to enhance model performance.
3. *Dataset*: Creation of a diverse and standardized dataset with 2,681 samples of defects from four fabric types and four defect types, ensuring comprehensive model training.
4. *Performance*: Demonstration of fast detection performance with inference times significantly lower than other models while maintaining high accuracy and other performance metrics.

The rest of the paper was structured as follows: In Sect. 1, we provided an introduction focusing on the challenges associated with fabric defect detection and the lack of a comprehensive public fabric defect dataset. The works within the same field were analyzed in Sect. 2. Section 3 discussed the importance and necessity of a comprehensive fabric defect dataset, detailing how we collected fabric defect samples, built our dataset, and created their masks. In Sect. 4, we introduced our AI model, which was designed specifically for detecting, classifying, and segmenting fabric defects with high accuracy and low complexity to expedite the detection process and build a very fast model. Various techniques were used to serve these purposes. Additionally, Sect. 5 discussed the training process and the evaluation metrics used. In Sect. 6, we comrformance of the proposed model with other state-of-the-art AI models. Finally, the paper was discussed and concluded in Sect. 7.

2 Related works

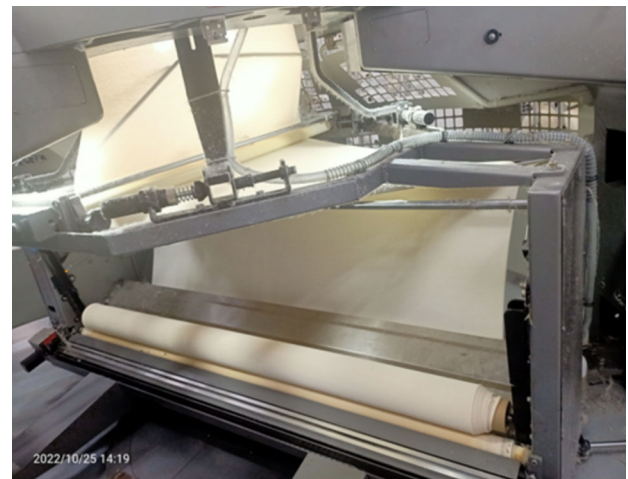
In the field of fabric defect detection, various approaches leveraging deep learning and advanced computational techniques have been proposed. For instance, Liu et al. [25] introduced an enhanced fabric defect detection model based on an improved version of YOLOv4, incorporating a novel spatial pyramid pooling (SPP) structure using SoftPool instead of MaxPool. Additionally, they applied contrast-limited adaptive histogram equalization (CLAHE) for enhanced image preprocessing. As a result, this method reportedly improved the mean average precision (mAP) by 6%, with only a minor 2% decrease in frames per second (FPS). However, despite the improvement in accuracy, the model's complexity raises concern about longer training times and slower inference, which could hinder real-time industrial applications. Similarly, Shakir and Topal [26] proposed an unsupervised method for fabric defect detection based on local spectra refinement (LSR), which analyzes spectral representations of fabric patterns without requiring prelabeled training data. By doing so, their approach outperformed existing techniques in accuracy, as tested on two datasets. They also introduced the ten fabrics dataset to validate their method further. However, their reliance on frequency-based techniques, such as Fourier transform and spectral analysis, makes the model highly effective for structured, repetitive fabric patterns. Nevertheless, their method assumes that defective regions behave as statistical outliers, which may not always hold true for fabrics with high natural variation or non-repetitive textures. In such cases, the system might mistakenly classify natural variations as defects or fail to detect actual defects. In addition, Yapi et al. [27] developed a learning-based method using local textural distributions in the contourlet domain. Utilizing redundant contourlet transform (RCT) and a mixture of generalized Gaussian distributions (MoGG), their model compares the statistical signatures of defect-free and defective fabric blocks to detect and localize defects. Moreover, the method demonstrated superior detection rates and reduced false alarms, especially on the TILDA dataset. However, it

remains computationally demanding due to its reliance on advanced textural analysis techniques. Furthermore, Liu et al. [28] proposed a multistage GAN-based approach to address the challenges posed by the diverse textures and defects encountered in real-world fabrics. Their framework integrates a deep semantic segmentation network with a generative adversarial network (GAN) to synthesize realistic defects and adapt to new fabric textures without manual annotations. The model was tested across various fabric types and demonstrated strong performance. However, the complex multistage training and synthesis process introduces significant computational overhead, which may limit its scalability in environments with limited resources. Additionally, Mei et al. [29] introduced an automatic fabric defect detection model based on a multi-scale convolutional denoising autoencoder (MSCDAE) network. The model employs Gaussian pyramid levels to reconstruct image patches and identify defects through pixel-wise predictions based on reconstruction residuals. This multi-scale approach, therefore, allowed the model to handle a wide variety of fabric types without manual intervention or labeled data. Despite its robustness and accuracy, the computational cost associated with multi-scale processing may restrict its application in real-time scenarios. Finally, Mohammed and Clarke [30] proposed a conditional generative adversarial network (cGAN) for fabric defect data augmentation. This model addresses the scarcity of comprehensive fabric defect datasets by generating synthetic defective fabric samples using a U-Net-based generator. Consequently, the generated data allow for the training of AI models to detect a broader range of defects, with precise control over defect characteristics such as type, shape, and location. Not only does this method enhance fabric defect detection accuracy, but it also shows potential for broader applications, such as generating synthetic medical imaging data. Finally, it is worth mentioning that the proposed research focused mainly on fabric defect detection, segmentation, and classification depending on a self-generated dataset.

3 Original dataset collection

To create an original dataset for fabric defect detection, we collected defective fabric samples from various weaving factories. This dataset includes a wide range of defect types, fabric textures, and color variations to ensure diversity and robustness. The collected samples were carefully annotated to support effective model training and performance evaluation. This approach allowed for a close examination of both the fabric weaving process and the weaving machines. Figure 1 illustrates a sample textile machine from one of the factories where data collection took place. While weaving machines across the different factories are generally similar in design, they are not identical in terms of technology and operational settings. Despite these variations, the data collection process was standardized to maintain consistency and ensure the robustness of the dataset.

Fig. 1 Fabric-knitting machine



In knitting factories, raw fabrics are typically single-colored (often woven in beige and later dyed into different colors in subsequent stages) and are woven at a speed of approximately one meter per minute. Defect inspection is carried out by experienced employees. When any defect is detected, the machine is stopped, necessary interventions are made to eliminate the causes of the defect, and then the machine is restarted. The main types of fabric defects commonly encountered during knitting are needle lines, holes, and stains (including oil and color stains) [31].

A. Needle lines

Needle lines are visible streaks or lines on fabric caused by issues with sewing machinery during manufacturing. They commonly result from factors such as damaged or dull needles, inconsistent stitching due to needle skipping, incorrect tension settings, thread irregularities, and poor fabric quality. Additionally, inadequate machine maintenance and operator errors, like improper sewing techniques, can contribute to these defects. Regular machine calibration, quality thread and fabric usage, proper tension adjustments, and skilled operators are key to preventing needle lines.

B. Holes

Holes are openings or gaps in the fabric's structure that reduce its quality and usability. They can result from mechanical damage due to excessive stress, abrasion, or improper handling during manufacturing and transportation. Poor material quality, manufacturing defects like improper stitching or cutting, and environmental factors such as moisture, sunlight, or pests also contribute to hole formation. Additionally, insect damage (e.g., moths), chemical exposure, and regular wear and tear in high-stress areas can weaken fibers, leading to holes. Proper handling, quality materials, and controlled storage conditions help prevent this defect.

C. Color and oil stains

Color and oil stains are common fabric defects that affect textile appearance and quality. Color stains result from unwanted dye or pigment transfer due to improper cleaning of dyeing equipment, mixing fabrics of different colors during processing, or exposure to uncontrolled dye sources. Preventive measures include regular equipment maintenance, color-based fabric separation, and strict quality control. Oil stains occur from contact with oils or greases during manufacturing, storage, transportation, or daily use. They can be minimized through clean machinery, proper storage, careful handling, and the application of stain-resistant finishes to fabrics.

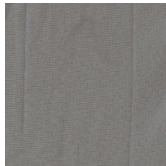
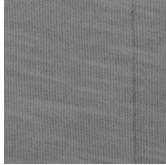
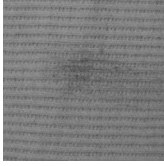
D. Roller marks

Roller marks are fabric defects appearing as lines or streaks caused by uneven pressure, texture variations, or contamination during manufacturing, processing, or storage. They often result from improper roller calibration, damaged or worn rollers, uneven surface temperatures, roller contamination, inadequate lubrication, material incompatibility, or pressure variations. To prevent roller marks, manufacturers should regularly inspect and maintain equipment, ensure proper roller calibration and lubrication, control surface temperature, and adjust processing parameters to match fabric properties. Consistent quality control helps minimize the occurrence of these defects.

A total of 2681 defective samples were collected from various knitting factories in Istanbul over a three-month period (Fig. 2). These samples encompass four fabric types and four defect types: needle lines, holes, color and oil stains, and roller markers (Table 1). The capture of these defect samples took place in a controlled lighting environment using a Nikon D5200 DSLR camera with a 24MP resolution, which was mounted at a specific height to achieve 4 PPMM (pixels per millimeter). This means that every 1 mm² in the actual fabric is represented by 4 × 4 pixels. All defect samples are in color and have a size of (256 × 256) pixels with a ratio of 4 PPMM. Detailed information about the defect samples is provided in Table 1.

After the dataset collection was completed, the process of generating labeled masks took place. For each image in the collected dataset, a corresponding mask with an identical size of (256 × 256) was created and labeled

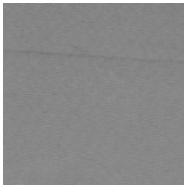
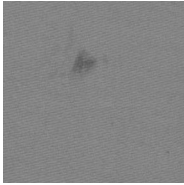
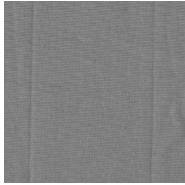
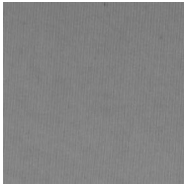
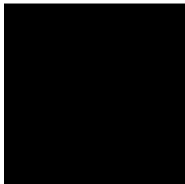
Fig. 2 Camera setup for collecting defect samples**Table 1** Collected dataset details

Defect name	Sample count on fabric 1	Sample image on fabric 1	Sample count on fabric 2	Sample image on fabric 2	Sample count on fabric 3	Sample image on fabric 3	Sample count on fabric 4	Sample image on fabric 4
Needle line	209		230		307		150	
Holes	240		194		18		335	
Color and oil stains	160		282		173		238	
Roller marks	145		0	Not available	0	Not available	0	Not available

manually. Each pixel in the generated masks was encoded with the corresponding class label (Table 2) for the related pixel in the original sample image. The mask labeling process took about two months.

Noteworthy that Table 1 demonstrates the statistics of the collected dataset regarding the number of samples for each defect type in all fabrics. However, since needle lines and roller marks appear as narrow lines, they have smaller and approximately constant areas compared to holes and stains, which have larger and variable-size areas. Figures 3, 4, 5, and 6 show the area distribution of holes and stains on fabric 1, fabric 2, fabric 3, and fabric 4, respectively. For example, consider Fig. 3B, which illustrates the area distribution of holes on fabric 1. We can see that the area of holes in about 140 samples is smaller than 0.5 cm^2 , about 90 samples contain holes with areas in the range of $0.5\text{--}1 \text{ cm}^2$, and the remaining 10 samples have holes with areas in the range of $1\text{--}1.5 \text{ cm}^2$.

Table 2 Defects classes

Defect type	Binary label	Example image	Related mask
Needle line	1		
Holes	2		
Stains	3		
Tight ends	4		
Un defected (clean)	0		

The defect area is calculated using the pixel density of the image. The pixel density is set to 4 pixels per millimeter (PPMM), meaning each pixel represents $\frac{1}{16}$ mm². The defect area is calculated by counting the number of pixels that represent the defect and converting this count into an area measurement [32, 33]. The conversion formula is as follows:

$$\text{Area}_{\text{cm}^2} = \frac{\text{Number of defect pixels}}{\text{PPMM}^2 \times 100} \quad (1)$$

where:

- Number of defect pixels is the count of pixels belonging to a particular defect class.
- PPMM is the pixel density (4 in this case).

Specifically, for each defect class, the area calculation is performed as follows:

For Class 2 defects (e.g., holes):

$$\text{Area}_{\text{cm}^2} = \frac{\text{Number of defect pixels}}{\text{PPMM}^2 \times 100} \quad (2)$$

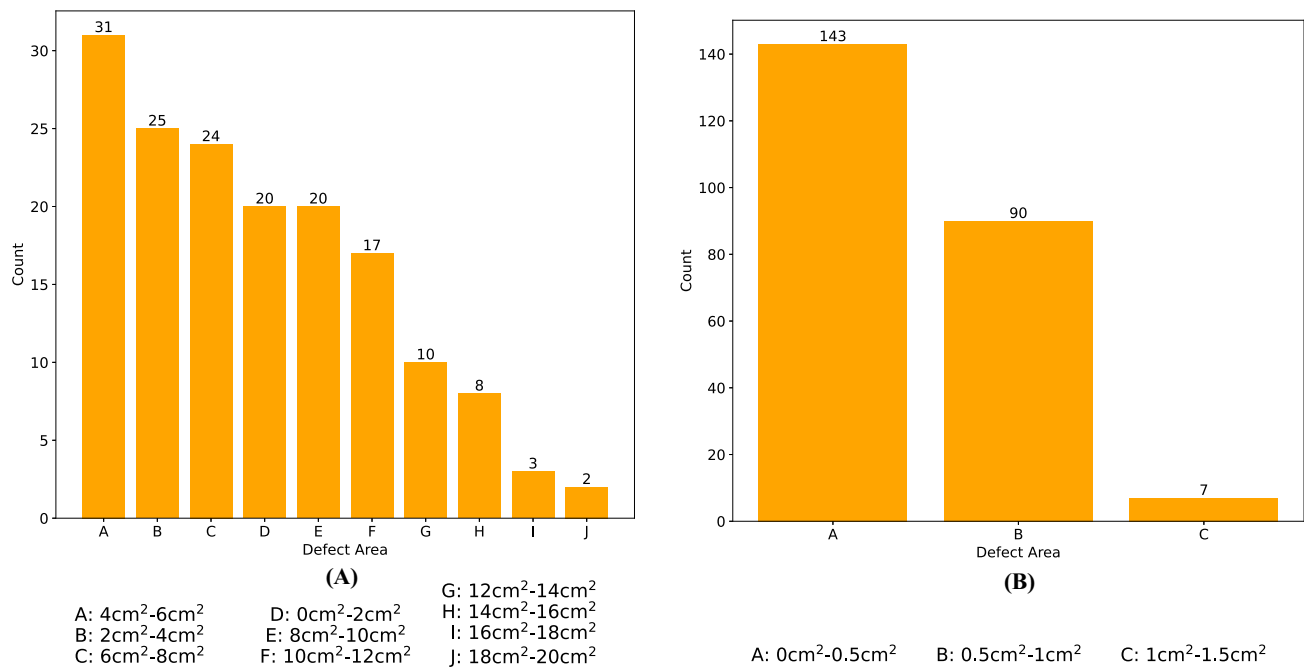


Fig. 3 Area distribution of holes and stains on fabric 1: **A** color and oil satins, **B** holes

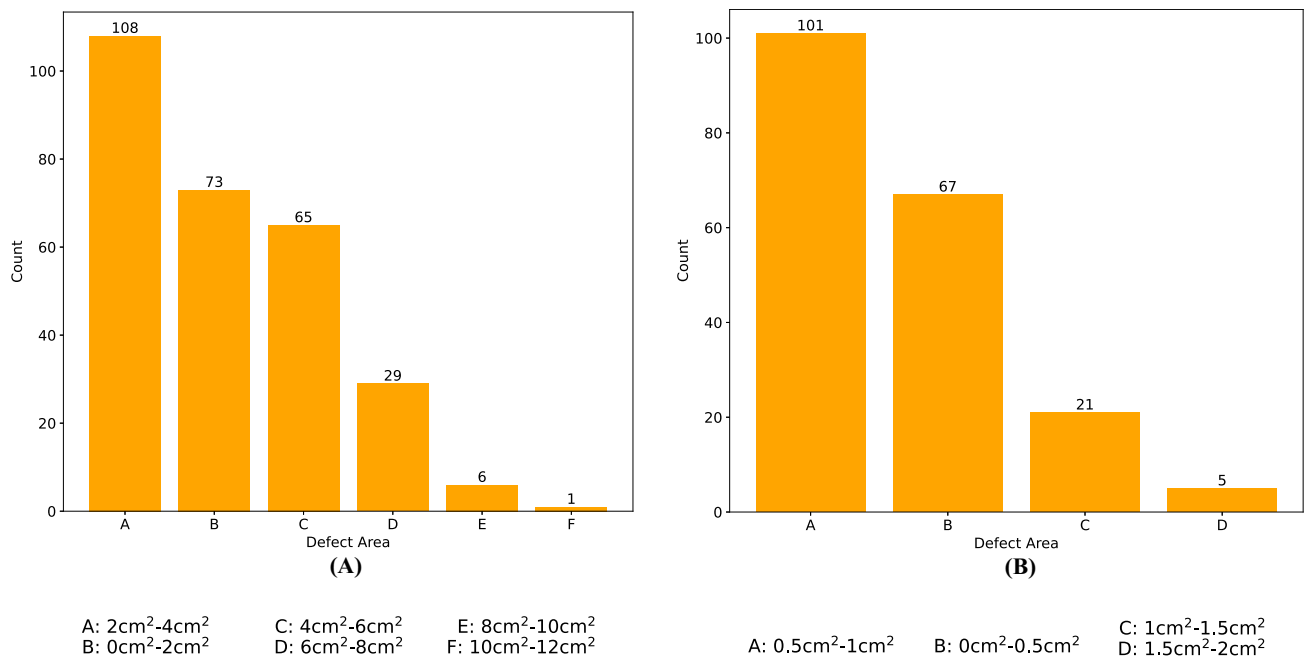


Fig. 4 Area distribution of holes and stains on fabric 2: **A** color and oil stains, **B** holes

$$\text{Area}_{\text{cm}^2} = \frac{\text{Count of pixels labeled as Class2}}{16 \times 100} \quad (3)$$

For Class 3 defects (e.g., oil and color stains):

$$\text{Area}_{\text{cm}^2} = \frac{\text{Count of pixels labeled as Class 3}}{16 \times 100} \quad (4)$$

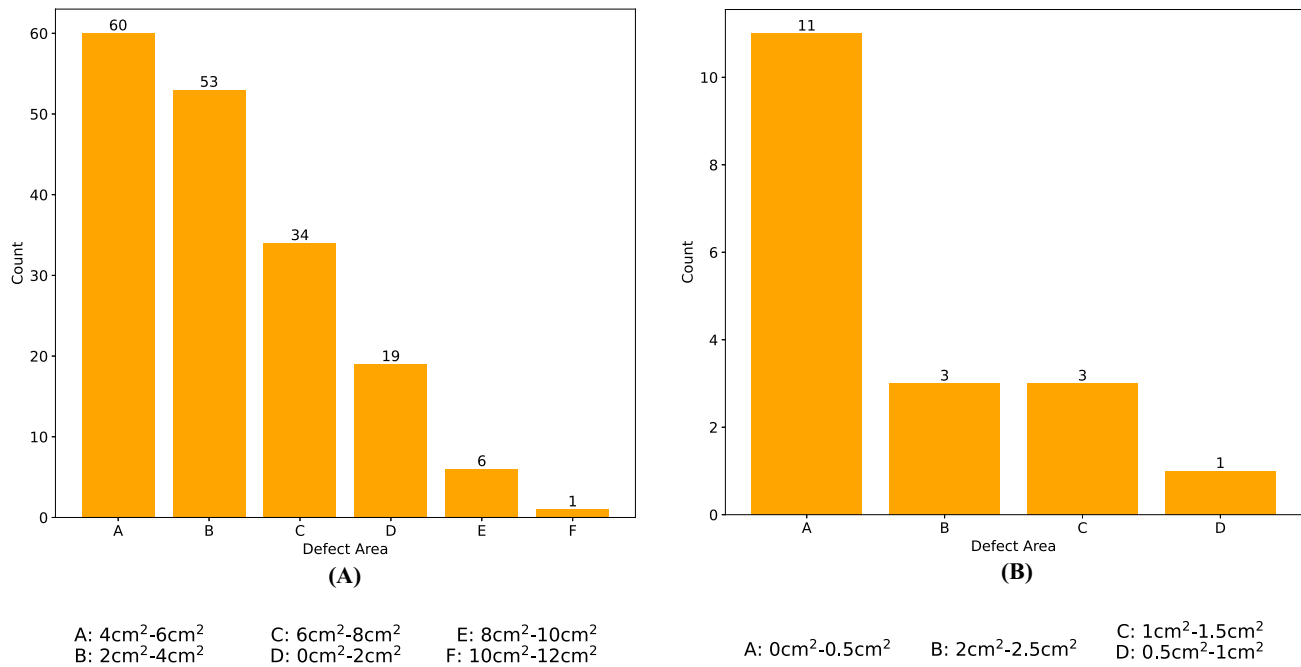


Fig. 5 Area distribution of holes and stains on fabric 3: **A** color and oil satins, **B** holes

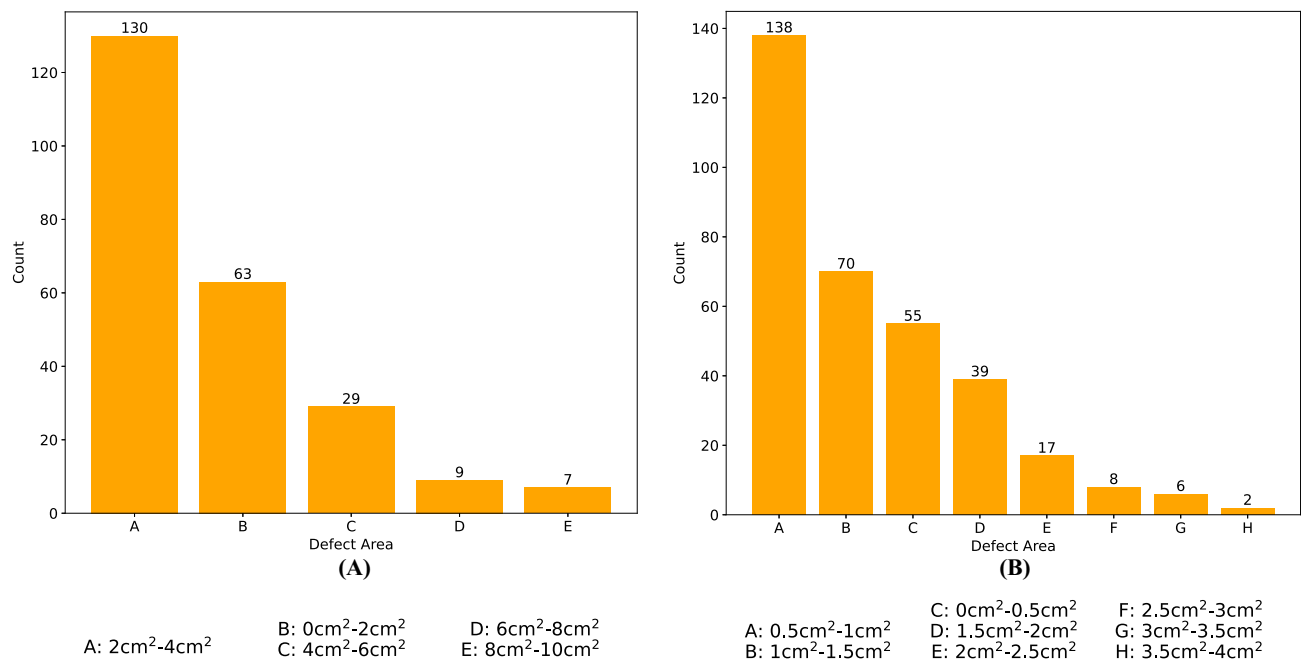


Fig. 6 Area distribution of holes and stains on fabric 4: **A** color and oil satins, **B** holes

The introduced ‘Semantic Segmentation’ artificial intelligence algorithm has provided labels in a mask format, details of which are presented in Table 1. ‘Semantic Segmentation’ was chosen because it fulfills a dual role of detecting defects and classifying the type of defect once detected. The provide domain labels in mask format would enable pinpointing of defect locations in terms of (X,Y) coordinates in the process of identification. Furthermore, since ‘Semantic Segmentation’ gives us additional information on size and shape about the detected defects, it would categorize the defects into significant, moderate, or negligible ones with respect to type and size. Once categorized, the machine operator can then make the necessary interventions.

The dataset, collected for the proposed model to ensure the diversity of data and consequent robustness, contains fabric samples from different factories covering a variety of types of fabrics characterized by different weaving patterns, textures, and colors. This dataset reflects realistic variations that can occur in the manufacture of the fabric, with defects of various sizes, shapes, and distributions appearing across the fabrics as found. In addition, data were acquired under controlled lighting conditions using a high-resolution camera so that the image quality would not suffer from variations in fabric properties. Data diversity contributes to generalization abilities in using different types of defects and different fabric conditions. Thus, it improves the efficiency and reliability of the model in practical applications.

4 Overview of the proposed CNN model designed for efficient fabric defect detection and segmentation

The primary goal of the proposed model is to achieve both high speed and accuracy, making it suitable for real-life fabric defect detection applications. In fabric manufacturing, human inspectors can typically examine fabrics at speeds of around 30 m per minute. However, due to the growing demand for higher production efficiency in modern factories, AI-based inspection systems are required to operate at higher speed as much as possible. Additionally, human inspectors often experience a decline in accuracy over extended work periods due to fatigue, reduced concentration, and subjective bias. In contrast, AI systems can maintain consistent accuracy and reliability without the effects of fatigue. This ensures that AI systems not only match human accuracy but also provide superior efficiency to keep up with fast-moving production lines, minimizing downtime and maximizing throughput. Unfortunately, most well-known semantic segmentation models, such as U-Net [34, 35], FPN [36], and LinkNet [37, 38], have complex architectures designed for high accuracy and applications like medical imaging. However, this complexity in their architecture makes them slow and unsuitable for real-time fabric defect detection applications.

The proposed model takes color images as input with a size of $(256 \times 256 \times 3)$ and produces detection masks of size $(64 \times 64 \times 1)$ pixels as output (Fig. 7). The input image initially passes through two sets of convolutional filters, with the first layer containing 32 filters of size (3×3) and a stride size of (2×2) to downsample the batch size by half without using an additional pooling layer to prevent any information loss and reduce complexity as much as possible. The second layer contains 32 filters of size (5×5) . This process is used to extract both small and larger features from the input image. It should be noted that the (3×3) and (5×5) filters mentioned refer to distinct convolutional layers, with the (3×3) filters applied in the first convolutional layer to capture fine details and the (5×5) filters in the second layer to extract larger contextual features from the input image.

The second stage integrates two ResNets acting in conjunction to extract additional features and enhance the detection capability. The first ResNet layer follows 64 convolutional filters of (3×3) size with a stride of (2×2) ; the second ResNet layer is followed by a set of 128 filters of size (5×5) . The two ResNets have compatible structures in terms of the number of sub-layers, which contain two sets of filters. The sizes of the convolution filters in the two sub-layers are (3×3) and (5×5) each for the first and second sub-layers, respectively. ResNet2, in its last stage, is again followed by one more set of 128 filters of size (3×3) .

The progression from two ResNet-type blocks was, by and large, devised to enhance feature extraction on various abstraction levels. This was important for fabric defect detection with precision. The first ResNet block is at the low- to mid-feature level of edge, texture, and simple defect patterns, while the second ResNet block processes these prior low- to mid-level features and tries to discern higher-level patterns characterized by complex arrangements impossible to discern from earlier layers. With this architecture, the model can very well characterize subtle and contrived defects. Besides, employing two lightweight ResNet-like modules instead of one deeper network helps to balance model performance with computation efficiency, realizing short inference periods while maintaining good detection accuracy.

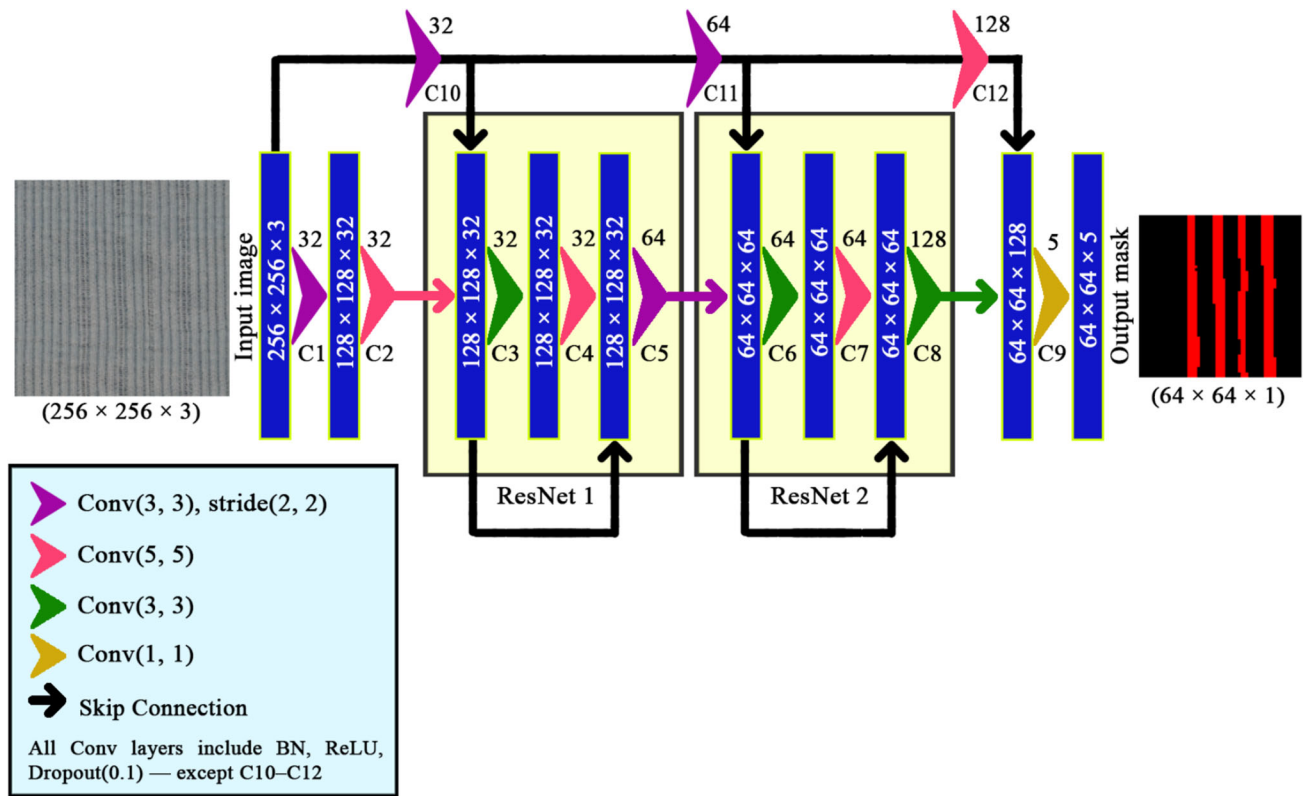


Fig. 7 Proposed model architecture

Unlike standard ResNet archetypes like ResNet-18 and ResNet-50, in this work, ResNet layers were used as lightweight, custom-tailored layers specifically designed for this application. These ResNet-inspired layers kept the residual connections to improve gradient flow and help learning; however, they are simplified based on much fewer layers and parameters. Thus, this custom design indicates decreased computational complexity and faster inference, making it suitable for real-time application tasks in fabric defect detection. The proposed model is built into two ResNet-inspired blocks with two convolutional layers of each block and of kernel sizes (3×3) and (5×5). The first ResNet block processes feature maps with 32 filters, the second with 64 filters, and finally with 128 filters, thereby gathering deeper features. Batch normalization (BN) and ReLU activation were applied after each convolution to stabilize training and improve convergence. A dropout layer (0.1) was also integrated into the network to solve generalization problems. It must be conceded that no upsampling operations were carried out, further reducing computational costs and ensuring the efficiency of defect segmentation. The overall architecture, shown in Fig. 7, is indicative of a balance between performance and speed that deem each blueprint appropriate for industrial fabric inspection tasks.

In the final stage, a set of 5 filters with a size of (1×1) is applied to the extracted features to produce a predicted mask for each class with a size of ($64 \times 64 \times 1$). To boost the prediction accuracy of the proposed model, the input image passes through 3 additional sets of convolutional filters, and the output of each set is skip-connected to the first layer of ResNet1, the first layer of ResNet2, and the last layer of feature extraction (C9), respectively. The padding is ‘same’ for all filters across the proposed model.

Each convolutional layer is followed by a batch normalization (BN), ReLU, and (0.1) dropout blocks, except for C10, C11, and C12. No upsampling blocks are used throughout the proposed model. The absence of an upsampling operation in the model design contributes to its speed and simplicity. Moreover, the resolution of the predicted mask is not a significant concern since it can be easily resized back to (256×256) afterward.

On the other hand, to ensure that the proposed model can accurately detect and segment defects, special techniques have been incorporated to develop a precise and powerful convolutional neural network (CNN) [39]. These techniques are explained in the following sequence:

A. Scheduled learning rate policy

Typically, when training deep learning models, an appropriate fixed learning rate is chosen (e.g., 1×10^{-4}), and this value remains constant throughout the training process [40, 41]. During model training, the loss value is calculated at the end of each epoch, and all of the model's parameters are updated based on this loss value and the learning rate ratio. However, usually during the training process, the loss value gradually decreases, and when the learning rate is kept constant, the number of parameter updates also decreases significantly. As a result, the learning process slows down considerably and may take a long time, and it may never reach the desired optimal points. Figure 8 provides a conceptual 2D cross-sectional illustration of the high-dimensional loss surface $J(\theta)$ to demonstrate the effect of learning rate schedules on convergence. This representation is for illustrative purposes only and does not imply any specific structural properties, such as axial symmetry, in the actual high-dimensional loss landscape. It is worth mentioning that the cost function $j(\theta)$ represents the error or the difference between the predicted values and the actual values in a machine learning model, while the model parameters (θ) represent the parameters of the model that are being optimized to minimize the cost function. These parameters are adjusted iteratively during the optimization process.

To overcome this problem, a scheduled learning rate method can be used. In the scheduled learning rate method, training starts with a learning rate equal to one-tenth of the predetermined maximum learning rate (Maxlr) in the first epoch. As the number of epochs increases, the learning rate also increases. When the training process reaches nearly thirty percent of the epoch count, the learning rate becomes equal to 'Max lr,' after which the learning rate gradually decreases again. In the final epoch, it drops to one-hundredth of the maximum learning rate value (Max lr/100). In this way, the model's learning process is accelerated, bringing it as close as possible to the optimal loss value.

B. Weighted cross-entropy loss function (unbalanced dataset problem)

In the context of data science projects, dealing with imbalanced datasets is a crucial issue. Most classification algorithms assume that training datasets are well-balanced [42]. The primary objective of these machine learning algorithms is typically to maximize the accuracy of predictions. However, these algorithms assume a balanced distribution among classes, which is often not the case in real-world datasets. In other words, one class may be represented by very few samples, while other classes may have a large number of samples. In this situation, classification decisions made by the trained AI model can pose challenges. The model is more likely to make incorrect predictions for classes with fewer samples because it has not been adequately trained for these classes [43].

Regrettably, the issue of imbalanced datasets is also present in the dataset created for this paper. As shown in Table 3, approximately 83% of the collected dataset represents the clean region (undefected area), while all

Fig. 8 Constant learning rate vs scheduled learning rate: (A) without scheduled learning rate policy, (B) with scheduled learning rate policy

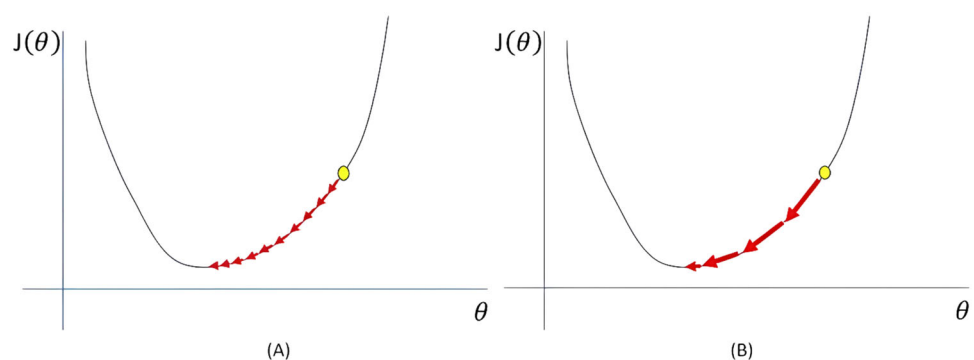


Table 3 Area percentage of each class in the dataset

No	Class name	Percentage (%)	Balanced weight	Class label
1	Un defected area	83.26	0.2402	0
2	Needle lines	03.63	5.5096	1
3	Holes	02.42	8.2644	2
4	Stains	10.25	1.6000	3
5	Roller markers	0.44	45.4545	4

other defect classes together account for only 17% of the complete dataset. Consequently, the AI model will primarily focus on learning to detect the clean region and may neglect learning to detect classes with lower proportions, the ‘Roller Marks’ class.

In multi-class classification problems with imbalanced datasets, we can use a weighted loss function to assign different levels of importance to each class during the training process. The weighted loss function encourages the model to prioritize underrepresented classes. The commonly used loss function for multi-class classification is the cross-entropy (log loss) function [44, 45].

$$L_{WCE} = - \sum [\omega_i \cdot (y_i \cdot \log(p_i) + (1 - y_i) \cdot \log(1 - p_i))] \quad (4)$$

where L_{WCE} is the weighted cross-entropy loss function, ω_i is the weight for class i . These weights are calculated based on the class imbalance, y_i is the true label for class i (1 if the sample belongs to class i , 0 otherwise), and p_i represents the predicted probability of the sample belonging to class i .

The class weights in the equation above are calculated using the following formula, which is based on the inverse of class frequencies [46]:

$$\text{Class weight for class}(C_i) = \frac{N}{C \times n_i} \quad (5)$$

where N represents the total number of samples (N) in the employed dataset, C is the total number of unique classes or labels in the dataset, and n_i refers to the number of samples in the specific class for which the class weight is being calculated.

5 Training process and obtained results

The proposed artificial intelligence model was constructed and trained using the Python tensor-flow library. Before applying any augmentation techniques, a subset of 300 samples was randomly selected and reserved as the test dataset. The remaining 2381 defective samples were augmented fourfold through rotation operations at four different angles (0°, 90°, 180°, and 270°). This rotation process not only changed the orientation of the samples but also increased the dataset size, requiring the model to learn more global features. The augmented dataset was divided into 80% for training and 20% for validation. Both the training and testing processes were carried out using Google Colab. The settings and parameters used for both training and testing are listed in Table 4.

When dealing with unbalanced datasets in machine learning, it is important to choose evaluation metrics that provide a more accurate reflection of the model’s performance, especially when the minority class is of interest, as in our case. Standard metrics like accuracy can be misleading in unbalanced datasets because they tend to be dominated by the majority class. Therefore, other performance metrics in addition to the accuracy metric will be used, including precision, recall, F1-score, and intersection over union (IoU).

Table 4 Training parameters and settings

Parameters	Values
Max_lr	8×10^{-4}
Epochs	150
Optimizer	Adam Optimizer
Beta 1	0.9
Beta 2	0.999
Loss	Weighted cross-entropy
Class weights	Class 0: 0.2402
	Class 1: 5.5096
	Class 2: 8.2644
	Class 3: 1.6000
	Class 4: 45.4545
Metrics	Accuracy, Recall, Precision and F1-Score
Last layer activation function	Softmax
Train subset size	7,619 (80%)
Validation subset size	1,905 (20%)
Test subset size	536
Input image size	$256 \times 256 \times 3$
Output mask size	$64 \times 64 \times 1$

Accuracy is a commonly used evaluation metric in machine learning that measures the overall correctness of a classification model. It calculates the proportion of correct predictions over the total number of predictions made. The formula for accuracy is as follows [39]:

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TF} + \text{FN} + \text{TP} + \text{FP}} \quad (6)$$

where (TP) represents true positives, while (FP), (TN), and (FN) stand for false positives, true negatives, and false negatives, respectively.

While accuracy is a straightforward metric, it may not be suitable for unbalanced datasets where one class significantly outweighs the others. In such cases, the accuracy metric can be misleading and may not accurately reflect a model's performance. This is because a model can achieve high accuracy by simply predicting the majority class for most samples.

Precision measures the proportion of true positive predictions among all positive predictions (both true positives and false positives). It is a useful metric when the cost of false positives is high or when the goal is to minimize the number of false positives [47, 48].

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \quad (7)$$

Recall measures the proportion of true positive predictions among all actual positive samples. It is particularly important when the goal is to identify as many true positive cases as possible, even at the expense of some false positives [49, 50].

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \quad (8)$$

The F1-score is the harmonic mean of precision and recall. It provides a balanced measure of a model's performance and is useful when you want to consider both false positives and false negatives. An F1-score closer to 1 indicates a good balance between precision and recall. For more clarity, F1-score formula is given as follows [51]:

$$F1 - \text{Score} = 2 \cdot \frac{(\text{Precision} \times \text{Recall})}{(\text{Precision} + \text{Recall})} \quad (9)$$

Intersection over union (IoU) is a common evaluation metric used in semantic segmentation and object detection tasks. It measures the overlap between the predicted segmentation mask and the ground truth mask. The IoU is calculated as the ratio of the intersection to the union of the predicted and ground truth masks [52]:

$$\text{IoU} = \frac{\text{Area of Intersection}}{\text{Area of Union}} \quad (10)$$

where:

- **Area of Intersection** is the number of pixels that are correctly predicted by the model (i.e., the pixels that belong to the defect class in both the predicted and ground truth masks).
- **Area of Union** is the total number of pixels that belong to the defect class in either the predicted or the ground truth mask.

During the 150-epoch training process, highly successful and very promising results were achieved, as shown in the Figures. 9, 10, 11, 12, 13, 14, respectively.

At the end of 150 epochs, it can be observed that the loss value dropped below 0.1 in both the training and testing phases. Furthermore, the accuracy measurement surpassed 96% in both the training and testing phases. On the other hand, we achieved very high scores for recall, precision, and F1-score, all of which were above 90%. These scores in the recall, precision, and F1-score metrics ensure that the trained model has a high detection capability for minor classes, which proves the importance and effectiveness of using class weights. Regarding the learning rate, it started at 8×10^{-5} in the first epoch, then increased to its maximum value of 8×10^{-4} around the 45th epoch. Afterward, it gradually decreased as the number of epochs increased, reaching 8×10^{-6} in the final epoch.

To test the trained model, five samples were randomly selected from the test subset and applied to the trained model. The model produced very accurate output masks, as shown in Fig. 15. The 'predicted mask' refers to the

Fig. 9 Training and validation loss

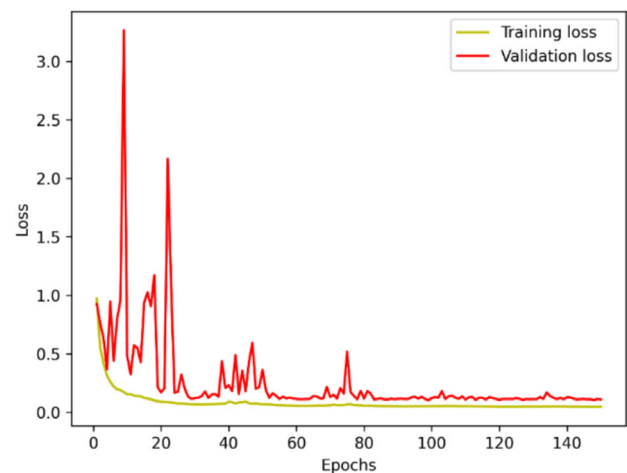


Fig. 10 Training and validation accuracy

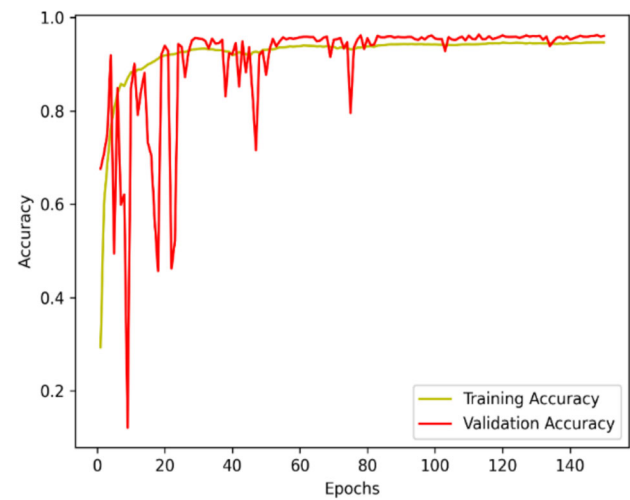


Fig. 11 Training and validation recall

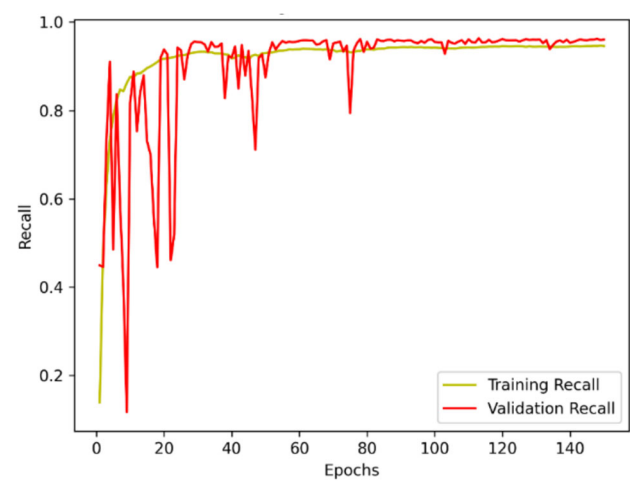


Fig. 12 Training and validation precision

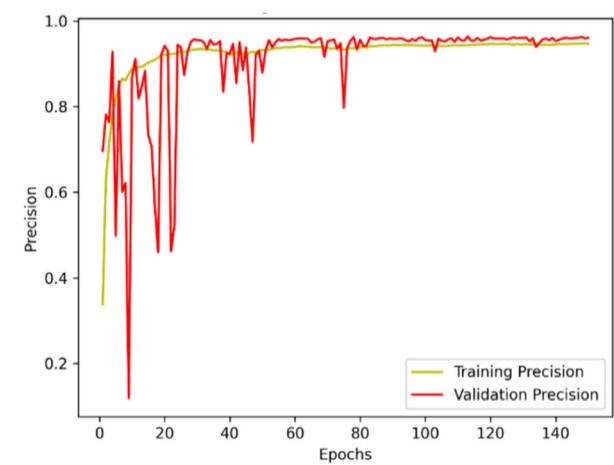


Fig. 13 Training and validation F1-score

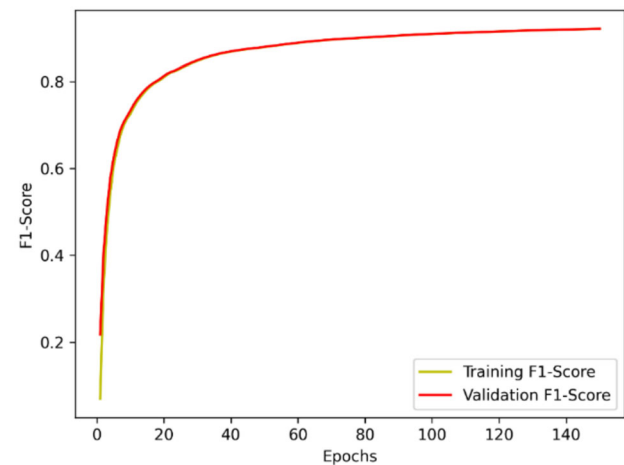
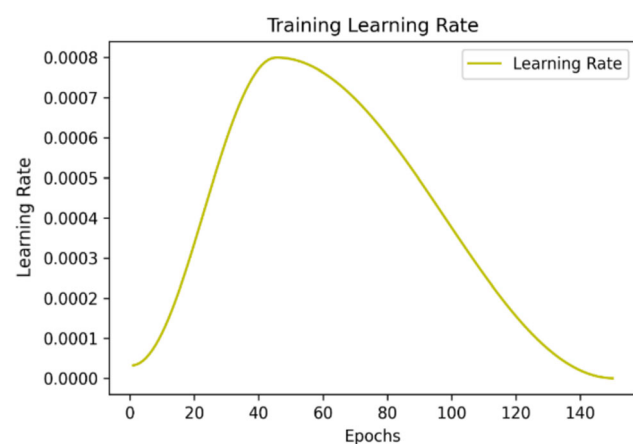


Fig. 14 Scheduled learning rate plot



mask generated by the trained model, and the ‘original mask’ refers to the manually labeled mask in the original dataset.

6 Comparison of the proposed model’s performance with other state-of-the-art AI models

To evaluate the performance of the proposed model compared to other state-of-the-art models such as SSG2, ResNet-50, YOLOv4, and multistage GAN (DeepLabV3 with ResNet-50), we trained and tested these models under the same conditions. The same parameters, datasets, and experimental setup were maintained to ensure a fair comparison. The datasets were divided consistently for training, validation, and testing phases, and all models were evaluated in the same Google Colab environment using identical hardware (Intel Xeon 2.20 GHz CPU and NVIDIA T4 GPU).

The comprehensive performance evaluation consists of two key components. The first is the performance metrics comparison, which includes metrics such as accuracy, recall, precision, F1-score, and IoU. These metrics are presented in Fig. 16, showcasing the performance of the proposed model relative to other state-of-the-art models. It is important to note that we could not evaluate performance metrics for YOLOv4 because it is primarily designed for object detection tasks, making it incompatible with our dataset, which focuses on image segmentation.

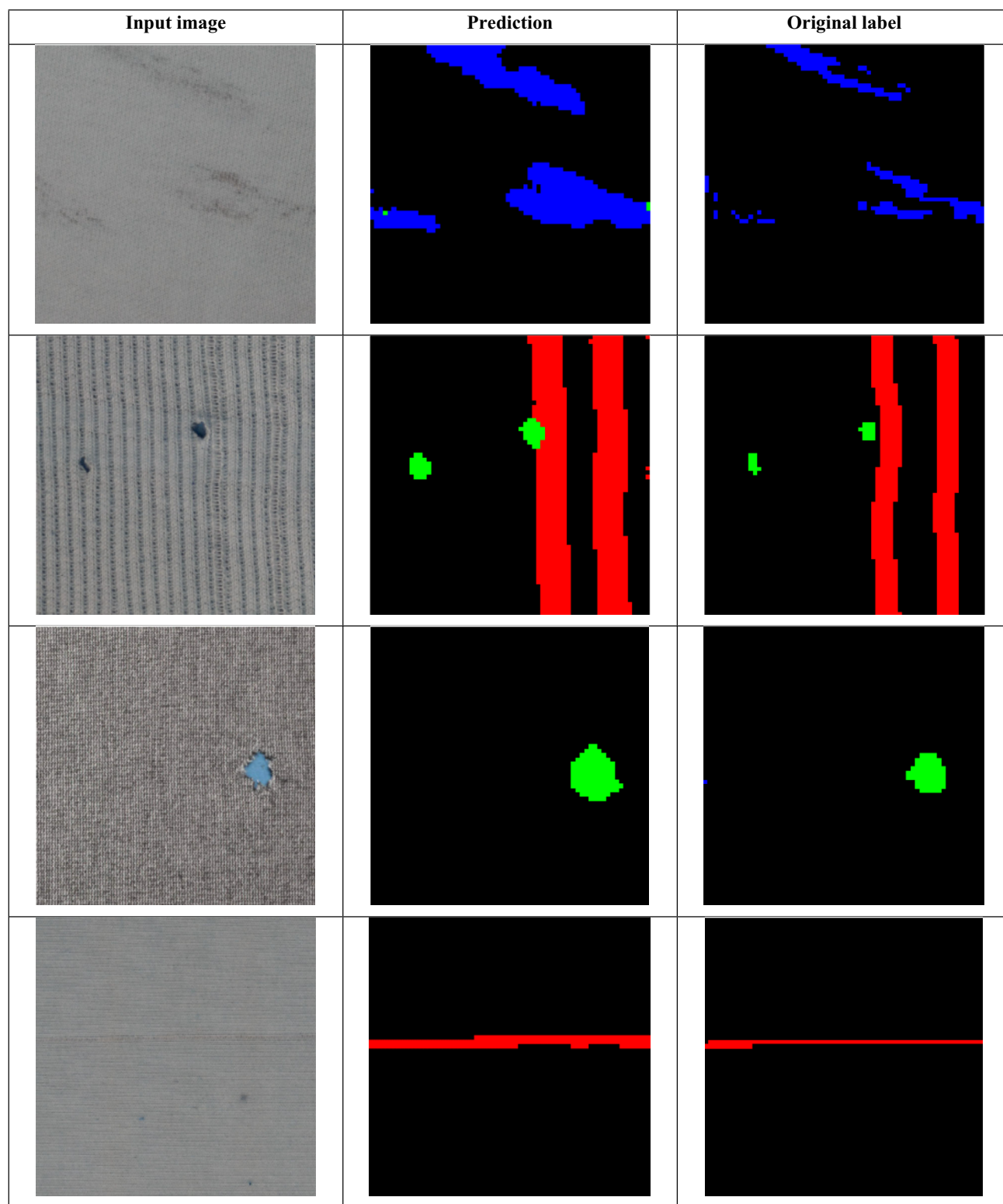


Fig. 15 Testing the trained model performance

The second component is the speed and complexity comparison, which covers aspects such as the number of parameters, inference time, model size, GFLOPs (Giga Floating Point Operations per Second—a measure of computational complexity indicating how many billion operations the model performs to process an input), and

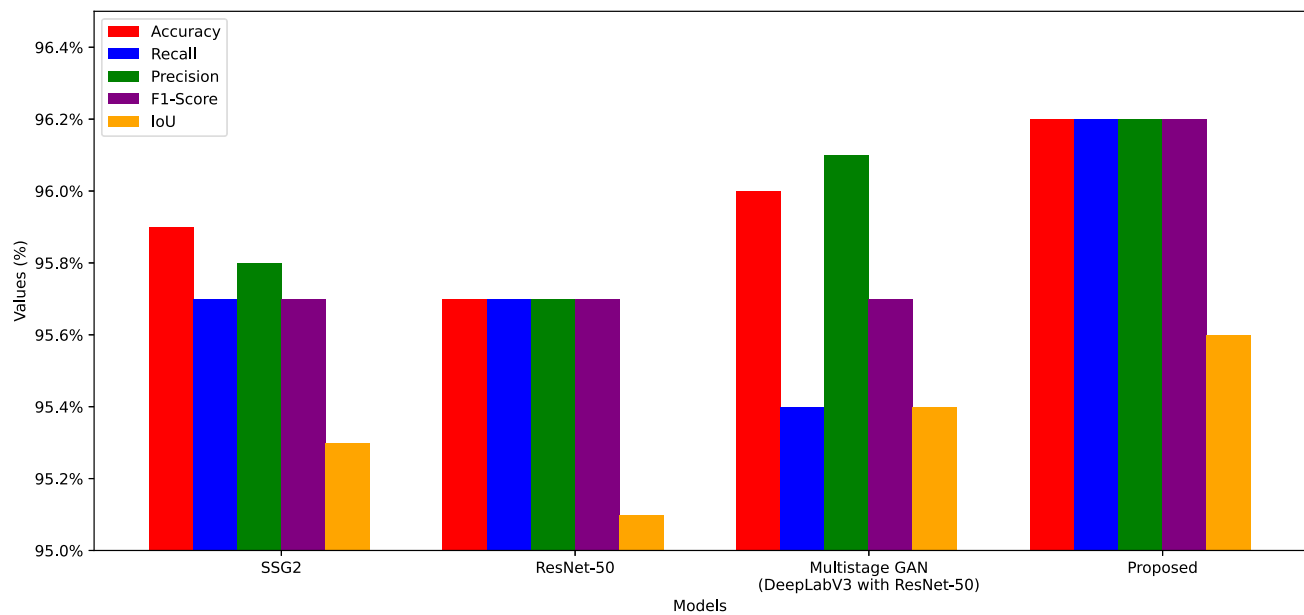


Fig. 16 Proposed model versus other state-of-the-art models (performance comparison)

Table 5 Proposed model vs. other state-of-the-art models (speed and complexity comparison)

Model	Parameters (millions)	Inference time (ms)	Size (MB)	GFLOPs	FPS (Frames Per Second)
SSG2	0.373	22.15	1.3	15	35
ResNet-50	35.27	47.61	134.55	92.64	21
YOLOv4 [25]	64	47.17	244	60–70	21.2
Multistage GAN (DeepLabV3 with ResNet-50) [28, 53]	23	40–80	90	20–40	12–25
Proposed	0.193	11.35	0.75	2.65	88.11

FPS (Frames Per Second). These details are summarized in Table 5, providing insights into the efficiency and computational demands of the models. In real-world industrial applications, determining an optimal inference speed requires in-field testing under actual production conditions, as it depends on factors such as hardware setup, software optimization, and inspection system design. However, since human inspectors work at speed around 30 m per minute, AI-based systems must be much faster than this to meet industrial needs, ensuring real-time processing while maintaining higher accuracy and consistency.

It is evident from the results that the proposed model outperformed all compared models across multiple metrics. The proposed model achieved the highest accuracy (0.962), recall (0.962), precision (0.962), F1-score (0.962), and IoU (0.956), with the lowest loss value (0.111). This superior performance highlights the model's robust learning capabilities and precise defect segmentation. Additionally, the proposed model demonstrated exceptional efficiency with only 0.193 million parameters and a compact model size of 0.75 MB, significantly lower than ResNet-50 (35.27 million parameters, 134.55 MB) and YOLOv4 (64 million parameters, 244 MB).

When comparing GFLOPs, the proposed model recorded the lowest value of 2.65 GFLOPs, indicating significantly reduced computational demands compared to ResNet-50 (92.64 GFLOPs), YOLOv4 (60–70 GFLOPs), and multistage GAN (20–40 GFLOPs). This reduction in computational complexity not only enhances processing efficiency but also contributes to faster inference speeds and lower energy consumption. Despite this low

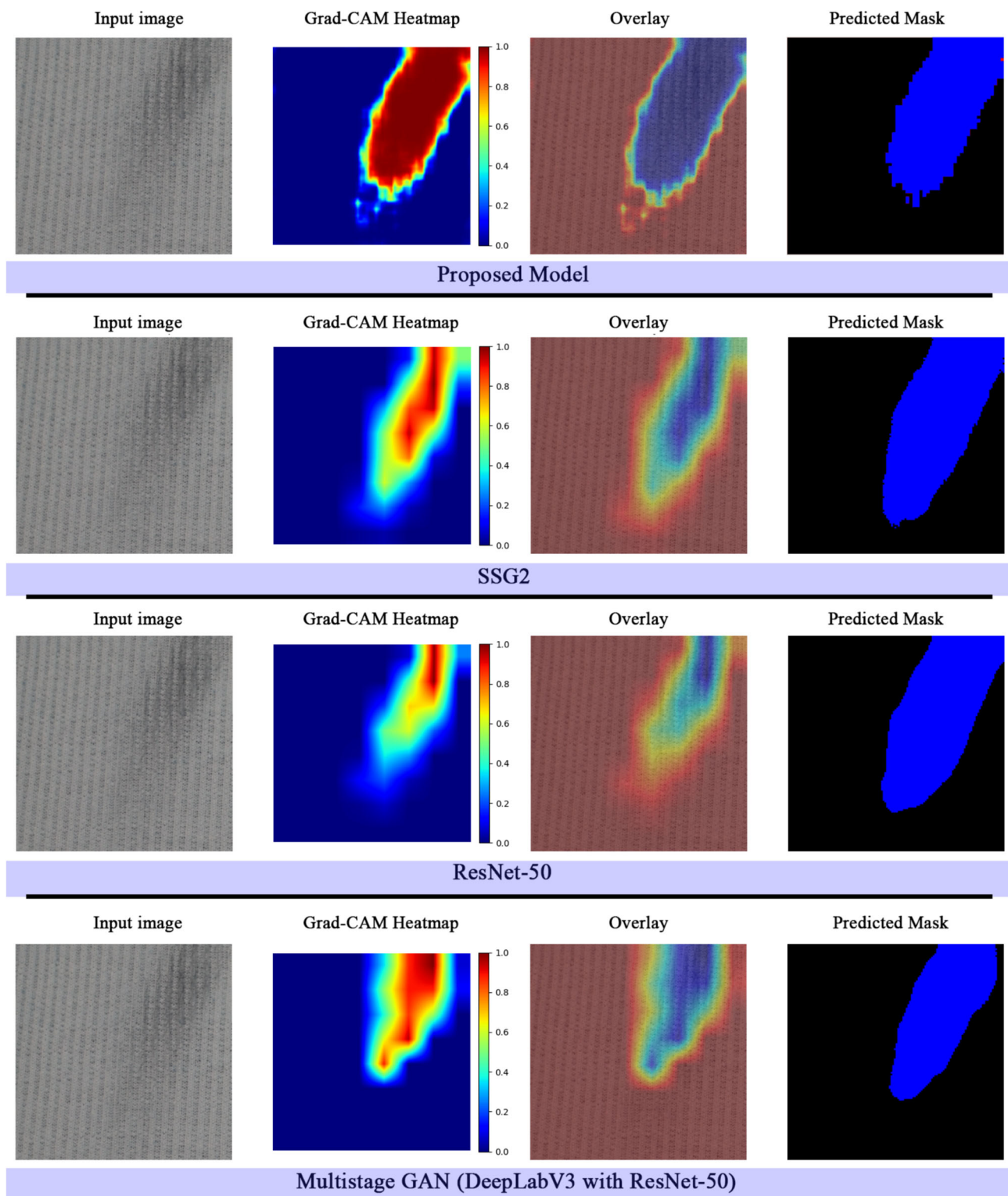


Fig. 17 Comparative visualization of attention regions and predicted masks across the proposed and SOTA models

GFLOPs, the proposed model delivered the fastest inference time (11.35 ms) and the highest FPS (88.11 FPS), confirming its exceptional capability to achieve high accuracy with minimal computational cost, making it highly suitable for real-time applications. To evaluate the interpretability and segmentation performance, a comparative visualization is presented in Fig. 17.

Last but not least, Fig. 17 illustrates a comparative visualization of the focus regions and predicted segmentation masks generated by four different deep learning models for fabric defect detection: the proposed model, SSG2, ResNet-50-based U-Net, and a multistage GAN (DeepLabV3 with ResNet-50). Each row presents the original input image, the corresponding Grad-CAM heatmap indicating the regions of highest model attention, the overlay of the heatmap on the original image, and the final predicted mask. As observed, the proposed model demonstrates precise localization of the defect with a well-defined activation region in the Grad-CAM and a clear segmentation boundary. SSG2 and the GAN-based model also localize the defect effectively but with relatively broader attention regions. In contrast, the ResNet-50 model shows scattered attention and less focused activation, which may lead to less accurate segmentation. These visualizations highlight the interpretability and discriminative power of each model, validating the superior localization capability of the proposed architecture.

7 Conclusion

Using artificial intelligence and image processing to detect defects in fabric is gaining increasing attention and importance. To achieve this goal effectively, it is essential to train the artificial intelligence model to be both fast and highly accurate. In pursuit of these objectives, the input images are downsampled to 1/4 of their original size ($256 \times 256 \rightarrow 64 \times 64$) without upsampling them to their original size; this reduces complexity and ensures a fast model. To improve the accuracy of detection and the performance of the proposed CNN model, techniques including ResNet, a scheduled learning rate policy, data augmentation, and a weighted cross-entropy loss function were used in the training stage. To facilitate the training process, a comprehensive, diverse, and standardized dataset was created. This comprised a total of 2,681 samples of defect, covering four fabric types and four defect types. The collected color samples are (256×256) pixels in size and have a ratio of 4 PPM (pixels per millimeter). The proposed CNN model was trained and tested using this dataset. After being trained for 150 epochs, the model exhibited both speed and a high level of performance. The loss value dropped below 0.1 in both training and testing, and the accuracy metric exceeded 96%. Recall, precision, and F1-score metrics were all above 90%, representing very high and successful results that guarantee the precision of the proposed model to predict minor classes. Comparing the proposed model to other state-of-the-art models such as SSG2, FPN, and LinkNet, it exhibited almost identical scores in loss, accuracy, recall, precision, and F1-score, and had the fastest prediction time. This aligns with the primary objective of developing this model, which is to create a solution for the detection of defects in fabric that is both fast and accurate. However, it is important to note that the proposed method has been primarily evaluated on plain fabrics. This presents a potential limitation regarding the model's effectiveness on more complex fabric textures, such as patterned, woven, or knitted fabrics, which introduce variations in patterns and colors that can potentially affect defect detection accuracy. Since the model was trained exclusively on plain fabrics, its performance on other textures may not be reliable.

Future work should focus on expanding the training dataset to include a diverse range of fabric textures. Collecting defect samples from various fabrics, such as silk, denim, and patterned textiles, would improve the model's robustness and generalization capabilities. Additionally, exploring techniques like transfer learning could enable the model to adapt to different textures with minimal retraining. These efforts are crucial to ensure the proposed model's versatility and real-world applicability in detecting fabric defects across different fabric types. It is believed that the proposed artificial intelligence model will offer a fast and effective solution for detecting defects in real-world applications.

Acknowledgements The authors would like to extend their appreciation to Ondokuz Mayıs University, and Northern Technical University for supporting this research.

Author's contribution Swash Sami Mohammed conceptualized and wrote the manuscript. Hülya Gökalp Clarke supervised the research. Sarmad Nozad Mahmood contributed to data collection and analysis. All authors reviewed and approved the final manuscript.

Funding No specific funding was received for this research.

Data availability The data are available from the corresponding author upon reasonable request. The data and material used in this research are available from the corresponding author on reasonable request.

Declarations

Conflict of interest The authors declare no conflict of interests.

References

- Li C, Li J, Li Y, He L, Fu X, Chen J (2021) Fabric defect detection in textile manufacturing: a survey of the state of the art. *Secur Commun Netw* 2021:9948808
- Rasheed A, Zafar B, Rasheed A, Ali N, Sajid M, Dar SH et al (2020) Fabric defect detection using computer vision techniques: a comprehensive review. *Math Probl Eng* 2020:8189403
- Kahraman Y, Durmuşoğlu A (2023) Deep learning-based fabric defect detection: a review. *Text Res J* 93:1485–1503
- Ahmad I, Hussain S, Mahmood SN, Mostafa H, Alkhayyat A, Marey M et al (2023) Co-channel interference management for heterogeneous networks using deep learning approach. *Information* 14:139
- Balhara S, Gupta N, Alkhayyat A, Bharti I, Malik RQ, Mahmood SN et al. (2022) A survey on deep reinforcement learning architectures, applications and emerging trends. *IET Communications*
- Yang J, Li S, Wang Z, Dong H, Wang J, Tang S (2020) Using deep learning to detect defects in manufacturing: a comprehensive survey and current challenges. *Materials* 13:5755
- Ieracitano C, Paviglianiti A, Campolo M, Hussain A, Pasero E, Morabito FC (2020) A novel automatic classification system based on hybrid unsupervised and supervised machine learning for electrospun nanofibers. *IEEE/CAA J Automatica Sinica* 8:64–76
- Zhang Z, Wen G, Chen S (2019) Weld image deep learning-based on-line defects detection using convolutional neural networks for Al alloy in robotic arc welding. *J Manuf Process* 45:208–216
- Pei X, HongZhao Y, Chen L, Guo Q, Duan Z, Pan Y et al (2023) Robustness of machine learning to color, size change, normalization, and image enhancement on micrograph datasets with large sample differences. *Mater Des* 232:112086
- Seçkin AÇ, Seckin M (2022) Detection of fabric defects with intertwined frame vector feature extraction. *Alex Eng J* 61:2887–2898
- Majety VD, Sharmili N, Pattanaik CR, Lydia EL, Zeebaree S, Mahmood SN et al (2022) Ensemble of Handcrafted and Deep Learning Model for Histopathological Image Classification. *CMC-Comput Mater Continua* 73:4393–4406
- Yaşar Çıklaçandır FG, Utku S, Özdemir H (2024) Determination of various fabric defects using different machine learning techniques. *J Text Inst* 115:733–743
- Mohammed SS, Clarke HG (2024) Advanced convolutional neural network approach for fabric defect detection. In: 2024 innovations in intelligent systems and applications conference (ASYU), pp 1–5
- Salem YB, Abdelkrim MN (2020) Texture classification of fabric defects using machine learning. *Int J Elect Comput Eng* 10:4390
- Mohammed SS, Clarke HG (2022) A hybrid machine learning approach to fabric defect detection and classification. In: International congress of electrical and computer engineering, pp 135–147
- Alkhayyat A, Abedi F, Bagwari A, Joshi P, Jawad HM, Mahmood SN et al (2022) Fuzzy logic, genetic algorithms, and artificial neural networks applied to cognitive radio networks: a review. *Int J Distrib Sens Netw* 18:15501329221113508
- Versaci M, Angiulli G, La Foresta F, Laganà F, Palumbo A (2024) Intuitionistic fuzzy divergence for evaluating the mechanical stress state of steel plates subject to bi-axial loads. In: Integrated computer-aided engineering, pp 1–17. <https://doi.org/10.3233/ICA-230730>
- Huang Y, Jing J, Wang Z (2021) Fabric defect segmentation method based on deep learning. *IEEE Trans Instrum Meas* 70:1–15
- Valero E, Forster A, Bosché F, Hyslop E, Wilson L, Turmel A (2019) Automated defect detection and classification in ashlar masonry walls using machine learning. *Autom Constr* 106:102846

20. Perez H, Tah JH, Mosavi A (2019) Deep learning for detecting building defects using convolutional neural networks. *Sensors* 19:3556
21. Lu B, Zhang M, Huang B (2022) Deep adversarial data augmentation for fabric defect classification with scarce defect data. *IEEE Trans Instrum Meas* 71:1–13
22. Zhang E, Li B, Li P, Chen Y (2019) A deep learning based printing defect classification method with imbalanced samples. *Symmetry* 11:1440
23. Aslam M, Khan TM, Naqvi SS, Holmes G, Naffa R (2019) On the application of automated machine vision for leather defect inspection and grading: a survey. *IEEE Access* 7:176065–176086
24. Abay NC, Zhou Y, Kantarcioglu M, Thuraisingham B, Sweeney L (2019) Privacy preserving synthetic data release using deep learning. In: *Machine learning and knowledge discovery in databases: European conference, ECML PKDD 2018, Dublin, September 10–14, 2018, Proceedings, Part I* 18, pp 510–526
25. Liu Q, Wang C, Li Y, Gao M, Li J (2022) A fabric defect detection method based on deep learning. *IEEE access* 10:4284–4296
26. Shakir S, Topal C (2024) Unsupervised fabric defect detection with local spectra refinement (LSR). *Neural Comput Appl* 36:1091–1103
27. Yapi D, Allili MS, Baaziz N (2017) Automatic fabric defect detection using learning-based local textural distributions in the contourlet domain. *IEEE Trans Autom Sci Eng* 15:1014–1026
28. Liu J, Wang C, Su H, Du B, Tao D (2019) Multistage GAN for fabric defect detection. *IEEE Trans Image Process* 29:3388–3400
29. Mei S, Wang Y, Wen G (2018) Automatic fabric defect detection with a multi-scale convolutional denoising autoencoder network model. *Sensors* 18:1064
30. Mohammed SS, Clarke HG (2024) Conditional image-to-image translation generative adversarial network (cGAN) for fabric defect data augmentation. *Neural Comput Appl* 36:20231–20244
31. Zhao Y, Hao K, He H, Tang X, Wei B (2020) A visual long-short-term memory based integrated CNN model for fabric defect image classification. *Neurocomputing* 380:259–270
32. Wu R, Wei H, Lu C, Liu Y (2025) Automatic and accurate determination of defect size in shearography using U-Net deep learning network. *J Nondestr Eval* 44:12
33. Jiangfei W, Lihua Y, Zhengguang Z, Mingyuan Y (2018) Accurate detection of a defective area by adopting a divide and conquer strategy in infrared thermal imaging measurement. *J Korean Phys Soc* 73:1644–1649
34. Wang L, Li R, Zhang C, Fang S, Duan C, Meng X et al (2022) UNetFormer: A UNet-like transformer for efficient semantic segmentation of remote sensing urban scene imagery. *ISPRS J Photogramm Remote Sens* 190:196–214
35. KrithikaAliasAnbuDevi M, Suganthi K (2022) Review of semantic segmentation of medical images using modified architectures of UNET. *Diagnostics* 12:3064
36. Pu B, Lu Y, Chen J, Li S, Zhu N, Wei W et al (2022) MobileUNet-FPN: A semantic segmentation model for fetal ultrasound four-chamber segmentation in edge computing environments. *IEEE J Biomed Health Inform* 26:5540–5550
37. Cai J-X, Mu T-J, Lai Y-K, Hu S-M (2021) LinkNet: 2D–3D linked multi-modal network for online semantic segmentation of RGB-D videos. *Comput Graph* 98:37–47
38. Ramasamy G, Singh T, Yuan X (2023) Multi-modal semantic segmentation model using encoder based Link-Net architecture for BraTS 2020 challenge. *Procedia Comput Sci* 218:732–740
39. Mahmood SN, Mohammed SS, Ismaeel AG, Clarke HG, Mahmood IN, Aziz DA et al. (2023) Improved malaria cells detection using deep convolutional neural network. In: *2023 5th International congress on human-computer interaction, optimization and robotic applications (HORA)*, pp 1–4
40. Yi D, Ji S, Park J (2021) An adaptive optimization method based on learning rate schedule for neural networks. *Appl Sci* 11:850
41. Abedi F, Ghanimi H, Sadeeq MA, Alkhayyat A, Kareem ZH, Mahmood SN et al (2023) Hybrid deep learning enabled load prediction for energy storage systems. *CMC-Comput Mater Continua* 75:3359–3374
42. Wang L, Han M, Li X, Zhang N, Cheng H (2021) Review of classification methods on unbalanced data sets. *IEEE Access* 9:64606–64628
43. Shi Q, Zhang H (2020) Fault diagnosis of an autonomous vehicle with an improved SVM algorithm subject to unbalanced datasets. *IEEE Trans Industr Electron* 68:6248–6256
44. Akil M, Saouli R, Kachouri R (2020) Fully automatic brain tumor segmentation with deep learning-based selective attention using overlapping patches and multi-class weighted cross-entropy. *Med Image Anal* 63:101692
45. Ho Y, Wookey S (2019) The real-world-weight cross-entropy loss function: modeling the costs of mislabeling. *IEEE access* 8:4806–4813
46. Alduaiji N, Algarni A, Abdalaha Hamza S, Abdel Azim G, Hamam H (2022) A lightweight CNN and class weight balancing on chest X-ray images for COVID-19 detection. *Electronics* 11:4008
47. Naidu G, Zuva T, Sibanda EM (2023) A review of evaluation metrics in machine learning algorithms. In: *computer science on-line conference*, pp 15–25
48. Qasim AN, Alani S, Mahmood SN, Mohammed SS, Aziz DA, Ata KIM (2024) Enhancing brain stroke detection: a novel deep neural network with weighted binary cross entropy training. *Revue d'Intelligence Artificielle* 38:777

49. Quazi S (2022) Artificial intelligence and machine learning in precision and genomic medicine. *Med Oncol* 39:120
50. Akhter R, Sofi SA (2022) Precision agriculture using IoT data analytics and machine learning. *J King Saud Univ Comput Inf Sci* 34:5602–5618
51. Song Q, Jiang H, Liu J (2017) Feature selection based on FDA and F-score for multi-class classification. *Expert Syst Appl* 81:22–27
52. Stodt J, Reich C, Clarke N (2023) Unified intersection over union for explainable artificial intelligence. In: *Proceedings of SAI intelligent systems conference*, pp 758–770
53. Tanveer M, Kim B, Hong J, Sim S-H, Cho S (2022) Comparative study of lightweight deep semantic segmentation models for concrete damage detection. *Appl Sci* 12:12786

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

Authors and Affiliations

Swash Sami Mohammed¹  · Hülya Gökalp Clarke¹ · Sarmad Nozad Mahmood²

✉ Swash Sami Mohammed

19211574@stu.omu.edu.tr; savaskrkk@gmail.com

Hülya Gökalp Clarke

hulya.gokalp@omu.edu.tr

Sarmad Nozad Mahmood

sarmad.nozad23@ntu.edu.iq

¹ Electrical and Electronics Engineering, Ondokuz Mayıs University, Samsun, Turkey

² Electronic and Control Engineering Department, Technical Engineering College – Kirkuk, Northern Technical University, Mosul, Iraq